

PROJECT 3

PCA USING SINGULAR VALUE DECOMPOSITION (SVD)

Dataset: Iris Dataset

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris

# -----
# 1. LOAD IRIS DATASET
# -----

iris = load_iris()
X = iris.data

print("=" * 60)
print("PCA USING SVD - IRIS DATASET")
print("=" * 60)

print("\nOriginal Dataset Shape:", X.shape)

print("\nFirst 5 Rows of Original Dataset:")
print(X[:5])

# -----
# 2. CALCULATE MEAN
# -----

mean = np.mean(X, axis=0)

print("\nMean of Each Feature:")
print(np.round(mean, 4))

# -----
# 3. CENTER THE DATA
# -----

X_centered = X - mean

print("\nFirst 5 Rows of Centered Data:")
print(np.round(X_centered[:5], 4))
```

```
# -----
# 4. CALCULATE COVARIANCE MATRIX
# -----

covariance_matrix = (
    np.dot(X_centered.T, X_centered)
    / (X.shape[0] - 1)
)

print("\nCovariance Matrix:")
print(np.round(covariance_matrix, 4))

# -----
# 5. FIND EIGENVALUES AND EIGENVECTORS
# -----

eigenvalues, eigenvectors = np.linalg.eigh(covariance_matrix)

# Sort eigenvalues from largest to smallest
index = np.argsort(eigenvalues)[::-1]

eigenvalues = eigenvalues[index]
eigenvectors = eigenvectors[:, index]

print("\nEigenvalues:")
print(np.round(eigenvalues, 4))

print("\nEigenvectors:")
print(np.round(eigenvectors, 4))

# -----
# 6. CALCULATE SINGULAR VALUES
# -----

n = X_centered.shape[0]

singular_values = np.sqrt(
    eigenvalues * (n - 1)
)

print("\nSingular Values:")
print(np.round(singular_values, 4))

# -----
# 7. CONSTRUCT V AND V^T
# -----

V = eigenvectors
```

```

Vt = V.T

print("\nV^T:")
print(np.round(Vt, 4))

# -----
# 8. CONSTRUCT S MATRIX
# -----

# IMPORTANT:
# X_centered has shape (150, 4)
# Therefore reduced S must be (4, 4)

S = np.diag(singular_values)

print("\nS Matrix:")
print(np.round(S, 4))

# -----
# 9. CALCULATE U
# -----

# U = X_centered * V * inverse(S)

U = np.dot(X_centered, V)

for i in range(len(singular_values)):

    if singular_values[i] > 1e-10:
        U[:, i] = U[:, i] / singular_values[i]

print("\nFirst 5 Rows of U:")
print(np.round(U[:5], 4))

# -----
# 10. VERIFY SVD
# -----

# X = U * S * V^T

X_svd = np.dot(
    np.dot(U, S),
    Vt
)

print("\nFirst 5 Rows of SVD Reconstruction:")
print(np.round(X_svd[:5], 4))

svd_error = np.mean(
    (X_centered - X_svd) ** 2

```

```
)

print("\nSVD Reconstruction Error:")
print(round(svd_error, 10))

# -----
# 11. EXPLAINED VARIANCE
# -----

explained_variance_ratio = (
    eigenvalues / np.sum(eigenvalues)
)

print("\nExplained Variance Ratio:")

for i in range(len(explained_variance_ratio)):

    percentage = (
        explained_variance_ratio[i] * 100
    )

    print(
        "PC", i + 1,
        "=",
        round(percentage, 2),
        "%"
    )

# -----
# 12. SELECT FIRST 2 PRINCIPAL COMPONENTS
# -----

k = 2

principal_components = V[:, :k]

print("\nFirst 2 Principal Components:")
print(np.round(principal_components, 4))

# -----
# 13. REDUCE DATA FROM 4D TO 2D
# -----

X_reduced = np.dot(
    X_centered,
    principal_components
)

print("\nReduced Dataset Shape:")
print(X_reduced.shape)
```

```
print("\nFirst 10 Rows of Reduced Dataset:")
print(np.round(X_reduced[:10], 4))

# -----
# 14. RECONSTRUCT DATA
# -----

X_reconstructed = np.dot(
    X_reduced,
    principal_components.T
)

# Add mean back
X_reconstructed = (
    X_reconstructed + mean
)

print("\nFirst 5 Rows of Reconstructed Data:")
print(np.round(X_reconstructed[:5], 4))

# -----
# 15. CALCULATE RECONSTRUCTION ERROR
# -----

reconstruction_error = np.mean(
    (X - X_reconstructed) ** 2
)

print("\nReconstruction Error using 2 Components:")

print(
    round(reconstruction_error, 6)
)

# -----
# 16. COMPARE DIFFERENT VALUES OF K
# -----

print("\nReconstruction Error for Different Dimensions:")

for k in range(1, 5):

    components = V[:, :k]

    reduced = np.dot(
        X_centered,
        components
    )

    reconstructed = np.dot(
        reduced,
```

```
        components.T
    )

    reconstructed = (
        reconstructed + mean
    )

    error = np.mean(
        (X - reconstructed) ** 2
    )

    print(
        "K =", k,
        " Error =",
        round(error, 6)
    )

# -----
# 17. PLOT PCA RESULT
# -----

plt.figure(figsize=(8, 6))

plt.scatter(
    X_reduced[:, 0],
    X_reduced[:, 1],
    c=iris.target
)

plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")

plt.title(
    "Iris Dataset - PCA using SVD"
)

plt.colorbar(
    label="Iris Class"
)

plt.grid(True)

plt.show()
```